10
Design A Web Crawler

A web crawler, also known as a robot or spider, is a useful piece of software that search engine information gatherers.

A web crawler is known as a robot or spider. It is widely used by search engines to discover new or updated content on the web. Content can be a webpage, an image, a video, a PDF file, etc. A web crawler starts by collecting a few web pages, then follows links on those web pages. A web crawler visits hyperlinks on each page it visits to find more web pages.

Figure 1

A crawler is used for many purposes:

- Search engine indexing. This is the most common use case. A crawler collects web pages to create a local index for search engines. For example, Googlebot is the web crawler for the Google search engine.
- Web archiving. This is the process of collecting information from the web to preserve data for long-term storage. Many national libraries run crawlers to archive websites. For example, the Library of Congress archives political websites.
- Web data mining. Web data mining is the process of using data to uncover patterns and insights. For example, a crawler can be used to download user profiles and social links from social media websites.
- Web monitoring. The crawler can be used to monitor copyright and trademark infringements. For example, a crawler can be used to scan pirated content on the web.

In this chapter, we design a web crawler that crawls HTML and new or updated content. We discuss how to scale a single-server crawler to multi-server distributed crawler. We also explore other interesting and related topics, such as how we prioritize web page collection, how we store collected data, how we avoid problems such as internet spam, and how we extend the data collection platform to other applications.

Step 1 - Understand the problem and establish design scope

The basic algorithm of a web crawler is simple:

1. Given a set of URLs, download all the web pages addressed by the URLs.
2. Extract URLs from these web pages
3. Add new URLs to the URL list to be downloaded. Repeat the process from step 1.

This simple version of a web crawler may work for small projects, for example, a school project. However, it is not practical to write a web crawler that can scale to billions of web pages. Building a web crawler at scale is a challenging task. Thus, we ask questions to clarify the requirements and gain a good understanding of what kind of system we need to build. Here are some example questions and answers that a candidate and interviewer might discuss when a crawling task is assigned.

Candidate: What is the purpose of the web crawler? Is it for search engine indexing, data mining, or something else?

Interviewer: Search engine indexing.

Candidate: How many web pages should the web crawler collect per month?

Interviewer: 1 billion pages.

Candidate: What content should be included? HTML only or other content such as PDF, JPEG and image files?

Interviewer: HTML only.

Candidate: Should we consider only articles or content of any type?

Interviewer: We are interested in HTML pages created from the internet.

Candidate: Do we need to store HTML pages crawled from the web?

Interviewer: Yes, up to five years.

Candidate: How should we handle web pages with URLs that are identical except for HTTP vs. HTTPS?

Interviewer: Treat them as two different web pages.

Candidate: What about URLs that point to the same web page, but with different parameters? For example, the URL www.example.com?id=1 and www.example.com?id=2 point to the same page if the parameter is used to identify a session. Should we consider such URLs?

Interviewer: No, we can ignore URLs with parameters.

Candidate: Do we need to consider URLs added to web pages?

Interviewer: Yes, we need to consider URLs added to web pages.

Candidate: How do we handle web pages that are unavailable due to network failure or server failure?

Interviewer: Retry later if a web page is unavailable.

Candidate: Should we respect robots.txt when crawling?

Interviewer: Yes, we should respect robots.txt.

Candidate: Do we need to store HTML pages crawled from the web?

Interviewer: Yes, we need to store HTML pages crawled from the web.

Candidate: How long do we need to store web pages? We can't store web pages forever because it is impractical to store web pages that are unchanged.

Interviewer: For simplicity, we can assume that we need to store web pages for up to five years.

Candidate: Should we consider HTTP headers? For example, should we use HTTP headers to determine if a web page has been updated?

Interviewer: No, let's keep it simple.

Candidate: What features should the web crawler support?

Interviewer: Support only HTTP and HTTPS.

Candidate: Should we support web page rendering? Web pages can be rendered by a web browser. For example, Chrome and Safari are web browsers that can render web pages. Rendering a web page may require the web crawler to execute JavaScript code. Should we support JavaScript code execution?

Interviewer: No, we don't need to support web page rendering or JavaScript code execution.

Candidate: Should we support extracting URLs from web pages?

Interviewer: Yes, we need to support extracting URLs from web pages.

Candidate: Should we support adding URLs to the URL frontier?

Interviewer: Yes, we need to support adding URLs to the URL frontier.

Candidate: Should we support other functionalities such as spam URL detection?

Interviewer: No, we don't need to support other functionalities such as spam URL detection.

Back of the envelope estimation

- 1 billion web pages are created per month. Assume that 1 billion web pages are created per month with the interviewer to be the same page.
- Assume 1 billion web pages are created per month.
- QPS: 1,000,000,000 / 30 days / 24 hours / 3600 seconds = ~385 pages per second.
- Peak QPS = 2 * QPS = 770
- Assume the average web page size is 500 KB.
- 1 billion pages * 500 = 500 TB storage per month. If you are unclear about binary and decimal units, see Back of the envelope estimation in chapter 2. Unless specified, we use the decimal system in this chapter. For example, 1 KB means 1000 bytes instead of 1024 bytes.
- Assume data collected for five years. The storage needed for five years is 500 TB * 12 months * 5 years = 30 PB. This much storage is a sizeable number. Assuming 3-way replication is used for durability, we need 30 PB * 3 = 90 PB storage.

Step 2 - Propose high-level design and get buy-in

After the requirements are clear, we move on to the high-level design. We first discuss some basic algorithms in web crawling. We, as a crawler, can start from a few seed URLs and use the web's hyperlink structure to crawl other URLs.

Figure 2

One an crawler each always crawlers are a crawler. For example, we start a web page from a university website on the assumption that all people in the university know each other.

Our crawler can also use the web page's hyperlinks to crawl other web pages. A crawler starts with a few web pages, then it parses the web page to find URLs to other web pages. The URLs to other web pages are added to the crawl list. This approach is known as breadth-first traversal. Breadth-first traversal is a good approach because a crawler can follow many different popular websites. Another way is to choose which link related to topics, such as sports or movies. The URLs to other web pages are added to the URL list. Unlike a web page, a URL can be in a specific format. We can use a URL parser to parse URLs.

URL Frontier

URL frontier is a data structure that stores URLs to be downloaded and already downloaded. The component that contains URLs to be downloaded is called URL frontier. We, as a URL list, or a FIFO queue. The URL frontier is a data structure that stores URLs to be downloaded. We, as a URL list, or a FIFO queue.

HTML Fetcher

The HTML fetcher downloads web pages from the Internet. These URLs are provided by the URL frontier.

DNS Resolver

To download a web page, a URL must be translated into an IP address. The HTML Downloader calls the DNS Resolver to get the corresponding IP address for the URL. For instance, URL www.wikipedia.org corresponds to IP address 198.35.26.96 as of 2019.

Content Parser

After a web page is downloaded, it must be parsed and validated because malformed web pages could cause problems in our system. We, as a crawler, will not waste any time on this. We, as a crawler, will waste any time on this. We, as a crawler, will waste any time on this.

Content Seen?

A web page is independent of the HTML structure, so it is inefficient for us to process a new web page. A web page's content may be different from another web page. We, as a crawler, can compare the content of a web page to other web pages. A web page's content is stored in a different content type. Content seen? Content seen? Content seen?